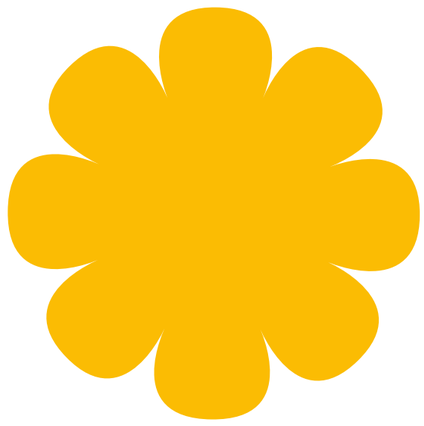
Google Skills

The solution: Tools Extending agent capabilities

How the architecture works



“In the context of ADK, a tool represents a specific capability provided to an AI agent, enabling it to perform actions and interact with the world beyond its core text generation and reasoning abilities.”

What tools enable:

- **Access real-time information:** Web search, weather APIs, stock prices
- **Perform calculations:** Execute code, run financial models, process data
- **Query databases:** Retrieve customer orders, product inventory, user profiles
- **Interact with external systems:** Send emails, book appointments, process payments
- **Take actions in the world:** Update records, trigger workflows, control devices

Simple example:

Python

```
def get_weather(city: str) -> dict:
    """Retrieves the current weather
for a specified city."""
    # In a real implementation, this
would call a weather API
    return {
        "status": "success",
        "report": f"Weather in {city}
is sunny, 25°C"
    }

# This Python function becomes a tool
when added to an agent
```

Reference: ADK Docs - What is a GTool?

Core concepts

1. What is a tool?

From ADK documentation:

"Technically, a tool is typically a modular code component—like a Python function, a class method, or even another specialized agent—designed to execute a distinct, predefined task."

Key characteristics:

1. **Action-oriented:**
   Tools perform specific actions for an agent, such as searching for information, calling an API, or performing calculations.
2. **Extends agent capabilities:**
   They empower agents to access real-time information, affect external systems, and overcome the knowledge limitations inherent in their training data.
3. **Execute predefined logic:**
   Crucially, tools execute specific, developer-defined logic. They do not possess their own independent reasoning capabilities like the agent's core LLM. The LLM reasons about which tool to use, when, and with what inputs, but the tool itself just executes its designated function.

Example—simple tool:

```python
def get_capital_city(country: str) -> str:
    """Retrieves the capital city for a given country."""
    capitals = {
        "france": "Paris",
        "japan": "Tokyo",
        "canada": "Ottawa"
    }
    return capitals.get(
        country.lower(),
        f"Sorry, I don't know the capital of {country}."
    )
```

What makes this a tool

- It's a **Python function** (modular code component)
- It performs a **specific task** (look up capital cities)
- It has **predefined logic** (the capitals dictionary)
- It **extends the agent's capabilities** (access to structured data)

Reference: ADK Docs - What is a Tool?

2. How agents use tools

From ADK documentation:

Agents leverage tools dynamically through mechanisms often involving function calling.

The process generally follows these steps:

1. **Reasoning:** The agent's LLM analyzes its system instruction, conversation history, and user request.
2. **Selection:** Based on the analysis, the LLM decides on which tool, if any, to execute, based on the tools available to the agent and the docstrings that describe each tool.
3. **Invocation:** The LLM generates the required arguments (inputs) for the selected tool and triggers its execution.
4. **Observation:** The agent receives the output (result) returned by the tool.
5. **Finalization:** The agent incorporates the tool's output into its ongoing reasoning process to formulate the next response, decide the subsequent step, or determine if the goal has been achieved."

Reference: ADK Docs - How Agents Use Tools

Example flow:

```
None
User asks: "What's the weather in Paris?"
    ↓
Step 1 - Reasoning:
    Agent analyzes: "User needs current weather data for Paris"
    ↓
Step 2 - Selection:
    Agent decides: "get_weather tool matches this need"
    ↓
Step 3 - Invocation:
    Agent calls: get_weather(city="Paris")
    ↓
Step 4 - Observation:
    Tool returns: {"status": "success", "report": "Paris is sunny, 20°C"}
    ↓
Step 5 - Finalization:
    Agent responds: "The current weather in Paris is sunny with a temperature of
20°C."
```

Key insight:
The LLM handles the reasoning and decision-making, while the tool handles the action and data retrieval. This separation is what makes agents powerful.

Visual representation:

```
None
graph TD
    A[User Request] --> B[1. Reasoning:<br/>Agent analyzes request]
    B --> C{2. Selection:<br/>Which tool?}
    C -->|Tool needed| D[3. Invocation:<br/>Call tool with args]
    C -->|No tool needed| E[Generate direct response]
    D --> F[4. Observation:<br/>Receive tool result]
    F --> G[5. Finalization:<br/>Formulate response]
    G --> H[Agent Response]
    E --> H

    style B fill:#e1f5ff
    style D fill:#ffeb99
    style F fill:#e1f5ff
    style G fill:#eeffee
```

The five-step process agents use to leverage tools

3. Tool types in ADK

From ADK documentation:

ADK offers flexibility by supporting several types of tools:

1. **Function Tools:** Tools created by you, tailored to your specific application's needs.
2. **Built-in Tools:** Ready-to-use tools provided by the framework for common tasks. Examples: Google Search, Code Execution, Retrieval-Augmented Generation (RAG).
3. **Third-Party Tools:** Integrate tools seamlessly from popular external libraries.”

Reference: ADK Docs - Tool Types in ADK

For this introductory lesson, we focus on:

1 Custom function tools

- Python functions you write
- Tailored to your specific business logic
- Example: `calculate_shipping_cost`, `lookup_order_status`
- Covered in: Part 3

2 Built-in tools

- Provided by ADK framework
- Production-ready, maintained by Google
- Examples: Google Search, code execution
- Covered in: Part 2

3 Agent-as-tool

- Use another specialized agent as a tool
- Enables delegation and specialized reasoning
- Example: Main agent calls a specialized “technical support” agent
- Previewed in: Part 4, Full coverage in: Future lessons

Comparison

Type	Complexity	Setup	Use when
Built-in	Low	Import and use	Need common capabilities (search, code execution)
Function Tools	Medium	Write Python function	Need custom business logic
Agent-as-Tool	Medium-High	Create specialized agent	Need specialized reasoning for subtasks

4. The `tools` parameter

From ADK documentation:

`tools` (Optional): Provide a list of tools the agent can use. Each item in the list can be:

- A native function or method (wrapped as a FunctionTool)
- An instance of a class inheriting from `BaseTool`
- An instance of another agent (`AgentTool`)”

How the LLM understands tools:

“The LLM uses the function/tool names, descriptions (from docstrings or the description field), and parameter schemas to decide which tool to call based on the conversation and its instructions.”

Code example:

Python

```
from google.adk.agents import LlmAgent

# Step 1: Define a tool function
def get_capital_city(country: str) -> str:
    """Retrieves the capital city for a given country."""
    capitals = {"france": "Paris", "japan": "Tokyo", "canada": "Ottawa"}
    return capitals.get(country.lower(), f"Sorry, I don't know the capital of
{country}.")

# Step 2: Add the tool to the agent
capital_agent = LlmAgent(
    model="gemini-2.5-flash",
    name="capital_agent",
    description="Answers questions about capital cities.",
    instruction="You are an agent that provides the capital city of a country.",
    tools=[get_capital_city]  # ✨ NEW: tools parameter
)
```

Reference: ADK Docs - Equipping the Agent: Tools

What to notice:

1. **Tools is a list:** `tools=[...]` - you can provide multiple tools
2. **Function passed directly:** ADK automatically wraps Python functions as `FunctionTool`
3. **LLM uses metadata:** Function name, docstring, and parameters guide the LLM's decision
4. **No manual wrapping needed:** Python ADK handles `FunctionTool` creation automatically

How the LLM sees this tool:

The LLM uses this schema to decide when and how to call the tool.

```
None
gTool name: get_capital_city
Description: "Retrieves the capital city for a given country."
Parameters:
  - country (str): A country name
Returns: str
```

5. Tools and session state (preview)

Connection to lesson 4: Managing state and memory

Tools can work together with session state to provide personalized, context-aware behavior. While the mechanics of accessing state in tools (`tool_context`) are covered in future lessons on callbacks and control, here's a preview of what's possible:

```python
def save_user_preference(preference_key: str, preference_value: str, ctx) -> dict:
    """Saves a user preference to session state.

    Args:
        preference_key: The preference name (e.g., "language", "theme")
        preference_value: The preference value (e.g., "Spanish", "dark")
        ctx: Tool context (provides access to session state)

    Returns:
        dict: Status message

    Note: The ctx parameter (tool_context) is covered in Lesson 6.
    """
    # Save to user: namespace for cross-session persistence
    ctx.session.state[f"user:{preference_key}"] = preference_value

    return {
        "status": "success",
        "message": f"Saved {preference_key} = {preference_value}"
    }
```

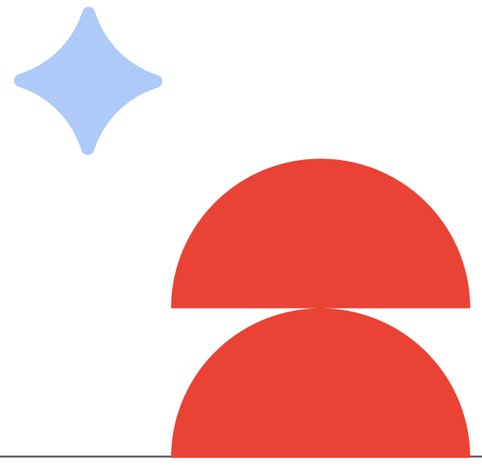
What this enables:

- **Personalization:**
  Tools can read user preferences from state
- **Context awareness:**
  Tools can check previous conversation state
- **Multi-step workflows:**
  Tools can save intermediate results to state
- **Cross-session memory:**
  Tools can use user: namespace for persistence

Example use case:

```python
# Turn 1: User sets preference
User: "Save my language preference as Spanish"
Agent calls: save_user_preference("language", "Spanish", ctx)
# Now state["user:language"] = "Spanish"

# Turn 2: Agent uses preference (via state templating from Lesson 4)
Agent instruction: "Respond in {user:language?English}"
# Resolves to: "Respond in Spanish"
```



Preview:

Full integration of tools with state management is covered in lesson 6, where you'll learn about `tool_context`, the `ctx` parameter, and advanced state coordination patterns.

Hands-on example

Building your first tool-enabled agent

What you'll build: A geography assistant that helps users learn about world capitals using a custom tool.

Step 1: Create the project

```shell
adk create geography_assistant
cd geography_assistant
```

What this does:

- Creates a new ADK project directory
- Sets up the basic agent structure
- Creates `agent.py` with default agent code

Step 2: Write the agent

Replace the contents of `agent.py` with:

Python

```
"""
Geography Assistant Agent
Demonstrates ADK's tools parameter with a simple custom function tool.

Reference: https://google.github.io/adk-docs/agents/llm-agents#tools
"""

from google.adk.agents import LlmAgent

# Step 1: Define a tool function
def get_capital_city(country: str) -> str:
    """Retrieves the capital city for a specified country.

    Args:
        country (str): The name of the country.

    Returns:
        str: The capital city name or error message.
    """
    # Simulated capital city database
    capitals = {
        "france": "Paris",
        "japan": "Tokyo",
        "canada": "Ottawa",
        "germany": "Berlin",
        "brazil": "Brasília",
        "australia": "Canberra",
        "india": "New Delhi",
        "mexico": "Mexico City"
    }

    # Look up the capital
    return capitals.get(
        country.lower(),
        f"Sorry, I don't have information about the capital of {country}."
    )
```

```
# Step 2: Create agent with the tool
root_agent = LlmAgent(
    model='gemini-2.5-flash',
    name='geography_assistant',
    description='Helps users learn about world geography.',
    instruction="""
    You are a geography assistant that helps users learn about world capitals.

    When a user asks about a capital city:
    1. Use the get_capital_city tool to find the answer
    2. Provide the information in a friendly, educational way
    3. You can add interesting facts if you know them

    If the tool returns an error message, politely tell the user you don't have
that information.
    """,
    tools=[get_capital_city]  # Provide the function as a tool
)
```

Code walkthrough

Lines 10–28: Define the tool function

- Function name is descriptive: `get_capital_city`
- Has type hints: country: `str -> str`
- Includes docstring explaining purpose and parameters
- Returns simple string (capital name or error message)

Lines 30–44: Create the agent

- Uses `LlmAgent` (from lesson 2–3)
- Adds new `tools` parameter with our function
- Instruction guides the agent on when and how to use the tool
- Explains how to handle both success and error cases

Step 3: Run and test

Shell

```
adk web
```

Navigate to `http://localhost:8000` in your browser.

Step 4: Test scenarios

Test 1: Basic capital query

You: `What is the capital of Japan?`

Expected behavior:

- Agent calls `get_capital_city(country="Japan")`
- Tool returns `"Tokyo"`
- Agent responds: “The capital of Japan is Tokyo.”

What to notice:

- Agent automatically selected the right tool
- Agent correctly extracted “Japan” as the country parameter
- Agent formatted the tool result into natural language

Test 2: Multiple capitals

You: `Tell me the capitals of France, Germany, and Brazil`

Expected behavior:

- Agent calls `get_capital_city` three times (once for each country)
- Tool returns: “Paris”, “Berlin”, “Brasília”
- Agent provides a formatted list

What to notice:

- Agent orchestrates multiple tool calls
- Agent understands it needs to call the tool for each country
- Agent synthesizes results into a cohesive response

Test 3: Unknown capital

You: `What is the capital of Iceland?`

Expected behavior:

- Agent calls `get_capital_city(country="Iceland")`
- Tool returns: “Sorry, I don’t have information about the capital of Iceland.”
- Agent politely relays this to the user

What to notice:

- Agent handles error messages gracefully
- Tool’s error message is user-friendly
- Agent doesn’t hallucinate an answer when data is unavailable

Test 4: No tool needed

You: `Tell me about geography`

Expected behavior:

- Agent responds without calling any tools
- Uses general knowledge about geography
- No tool invocation occurs

What to notice:

- Tools are called only when needed
- Agent uses its reasoning to determine tool necessity
- Not every query requires a tool

Key takeaways

Core concepts:

- **Tools extend agent capabilities** beyond LLM knowledge, enabling real-world actions and data access
- **Agents use tools through five steps:** Reasoning → Selection → Invocation → Observation → finalization
- **Three main tool types:** Built-in tools (ready-to-use), function tools (custom), agent-as-tool (delegation)

How tools work:

- **LLM selects tools based on** function name, docstring, and parameter schema
- **Tools are just Python functions** that ADK wraps automatically
- **Tools return results** that agents incorporate into their responses
- **Orchestration happens automatically** through ADK's agent loop

The `tools` parameter:

- **Added to `LlmAgent`** - `tools=[function1, function2, ...]`
- **Functions become tools automatically** - No manual wrapping needed in Python
- **LLM decides when to use each tool** - Based on conversation context and instructions

Best Practices:

- **Descriptive function names** – Use `get_capital_city` not `lookup` or `get_data`
- **Clear docstrings** – Explain what the tool does and when to use it
- **Type hints** – Help ADK generate proper schema for the LLM
- **Simple return values** – Start with basic types before complex structures

Agent Behavior:

- **Tools are optional** - Agents call tools only when reasoning determines they're needed
- **Multiple calls possible** - Agents can call the same tool multiple times or different tools
- **Natural integration** - Tool results are seamlessly incorporated into responses
- **Error handling** - Tools should return clear error messages for failure cases